The Imposed and Emergent Pieces of Convolution Under an Energy Budget

Vasili Gavrilov*

2026

Abstract

Which pieces of a convolution must be *imposed* as priors, and which *emerge* on their own under a resource budget? We grow a network's topology from an **exhaustive, fully-connected seed under an evolutionary energy budget** that pays for every connection, using the search as a *lens* on that question rather than as an optimizer, and map the boundary honestly. Sparse task-relevant connectivity emerges cleanly, and weight sharing is *adopted* when translation invariance rewards it; but the compact aligned local filter emerges only once mutation acts on the shared feature the way real genetics does, not on a single edge, and composition on the channel axis does not emerge from the task label alone at all. On top of the priors a ConvNet hardwires (locality, translatability), the free dimensions then organize to the task: compact fields and a depth that grows with (though overshoots) the required receptive field.

What is new: -the sharing/energy *decoupling*, our one mechanistic result: once weights are shared, connection count no longer gradients parameter count, so a compact filter *cannot* emerge under per-connection pruning, and does only once mutation acts on the whole shared feature, which restores the missing energy gradient; -the resulting *map* of convolution's imposed-versus-emergent pieces from an exhaustive seed, negatives kept, including the boundary where emergent *composition* needs supervision, a characterization that treats the search as a lens, not an optimizer; -a control confirming the directed search does real work: at a matched budget it finds a *tidier* (more energy-efficient) filter than random sampling at *equal task accuracy*, which *reconciles* the near-null our companion crossover study found on a flat benchmark landscape. Much of the rest reproduces textbook inductive-bias facts (weight sharing is data-efficient, receptive field $\approx$ depth), verified with a fair baseline rather than claimed as new; path-independence (the same structure from a minimal seed grown up) is a robustness check, with grow-and-prune sparse training (SET, RigL) its near neighbor. Every number regenerates from one `make` at the per-section seed counts noted in the text (via `SEEDS`/`NTR`); the dead-ends are kept.

1 Introduction

This paper grows a network's architecture instead of searching for it (Figure 1): start from an exhaustive, fully-connected seed, put a cost on every connection, and see which parts of convolution survive. The question is not "can a search find a good architecture" but "what structure emerges on its own under an energy budget, and what does not," in the best case rediscovering the convolutional receptive field.

The idea has a long history. A 1997 thesis argued that heuristics built into the architecture by hand are a *necessity* for a trainable network [11]; the ambition to instead let that structure *emerge* from a genetic-algorithm search came in 2002, after the author had worked with genetic

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning, including a genetic-algorithm optimizer built for industrial use in 2002. Reference implementation: `https://github.com/Anode1/SMBPANN`, 2001–2015–2026.

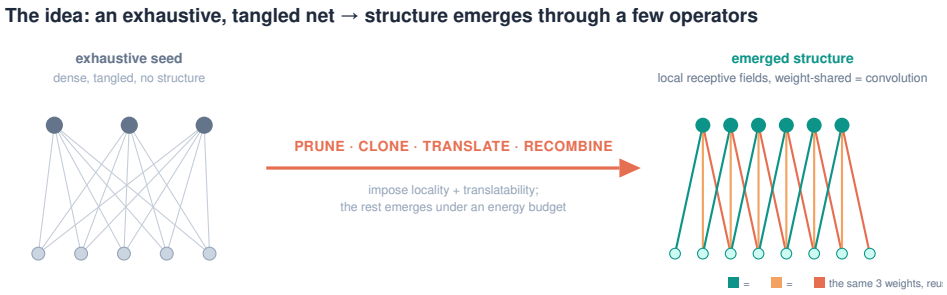

Figure 1: The idea. An exhaustive, tangled network becomes an ordered convolution through a few structural operators. We impose the domain's real symmetries (locality, translatability); the operators, prune, clone, translate, recombine, are biological analogs we study individually and then *chain* (find a block, then clone and translate it, Section 3.4). Chaining in the fourth, where the search must *discover* the decomposition, does not cleanly emerge (Section 3.5).

algorithms in industry, and the project, created in 2015, is first implemented now in pure C. A companion study came at the same idea from the search side and reached a firm negative: on NAS-Bench-101 no crossover meaningfully beats random search, because good cells are common and the gap to the optimum sits inside the benchmark's noise (Section 2.4 returns to this under an energy budget, where the picture differs).

Contribution, stated honestly. Our one mechanistic result is the sharing/energy *decoupling*: a compact filter cannot emerge under per-connection pruning, because sharing decouples connection count from parameter count, and emerges only once mutation acts on the whole shared feature (as a gene acts on every instance it builds, not on one edge). Around it we draw a *map*, using the energy GA as a *lens* rather than an optimizer, of which pieces of convolution are imposed priors and which emerge from an exhaustive seed, with the boundary where emergent composition needs supervision marked rather than hidden. This use of evolution, and this map, is distinct from gradient pruning (Optimal Brain Damage; the Lottery Ticket), from DARTS' gradient-relaxed supernet, and from NEAT's grow-from-minimal, which augment rather than prune. Much of the rest, weight sharing is data-efficient; receptive field $\approx$ depth, reproduces known inductive-bias facts, which we verify carefully rather than claim as new.

Method in one paragraph. A one-hidden-layer network whose connectivity is a binary mask is evolved by a genetic algorithm from an all-ones (dense) seed. Fitness is validation accuracy on a scarce training set; the objective is to minimize energy (the number of connections, or free parameters) subject to accuracy $\geq$ target. Mutation is mostly-prune, rarely-grow, an annealing schedule, in which a prune is a downhill move in energy and a rare added connection is an uphill thermal fluctuation. All numbers below are means over independent seeds; each experiment is a self-contained C99 probe.[1]

2 What emerges, and what does not

2.1 Pruning: feature selection emerges; convolution does not

Under an energy budget the search prunes a dense connectivity mask to a sparse one, and on a task whose label depends on a few specific inputs it lands **90%** of the surviving connections

[1]The identical training loop is $\sim$130$\times$ slower in naive Python/NumPy than in C (measured); a compiled path (Numba, JAX) closes most of the gap, so the edge is over interpreted Python, not fundamental.

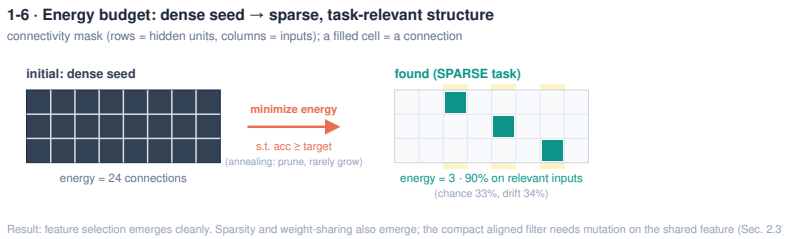

Figure 2: Sec. 2.1. An energy budget prunes a dense connectivity seed to a sparse mask whose surviving connections land on the task-relevant inputs. Feature selection and sparsity emerge; the compact aligned filter needs mutation on the shared feature to emerge (Sec. 2.3).

on the task-relevant inputs (chance 0.33, no-selection drift 0.34). Feature selection, discovering *which* inputs matter, emerges cleanly (Figure 2). What does *not* emerge is the aligned local receptive field: at comparable density the surviving connections are no more in-window than chance, so the search finds *a* sparse subnetwork, not *the* convolutional one, exactly the outcome of the pruning literature (sparse subnetworks, not convolutions).

2.2 Imposing locality: a compact filter emerges, sharing is adopted

Locality is not optional; a bounded receptive field is a constraint of the problem (information is local), and every ConvNet imposes it. When we impose only that, each hidden unit sees a *contiguous* window, and a compact local field emerges directly: the width anneals to a mean of $\approx K$ (the kernel size), the compact filter the energy budget alone could not produce. A weight-tying gene (tie weights by within-window offset, i.e. translation invariance) is *adopted* by 0.89 of the population; but its accuracy benefit is within noise, and the shared arm stays wider, because shared energy charges only the maximum width, so the mean has no gradient. The pieces of convolution keep emerging separately; the clean compact *shared* whole does not dominate.

2.3 The compact filter, and the mutation that finds it

There is a subtlety in what "an energy budget" means here, and it is, we think, the paper's one real insight. Once weights are shared, removing a single connection no longer reduces the parameter count, that offset is still used by other positions, so a mutation acting on one edge has no energy gradient to tighten the kernel. The right unit of mutation is not the edge but the shared feature.

The compact filter does emerge, under the right mutation. Biological mutation is global, not local: a change to a gene acts on every instance of the mechanism it builds (all the "similar blocks and duplicated edges"), not on one edge. So for a weight-shared filter the natural unit of mutation is the shared *offset*, all the connections that reuse that weight; flipping a whole offset removes all its duplicated edges in one move and reduces the parameter count, which restores the energy gradient a single-connection edit cannot give. Under this grouped mutation the kernel contracts: the surviving tap count falls to 2.6 (near $K = 3$) over 16 seeds, and a width penalty sharpens it to contiguity 0.80 on the generative offsets. It is not a perfect $K = 3$ kernel (span ≈ 4), and the offset genome makes the connectivity translation-invariant by construction, but the compact filter largely emerges once mutation acts on the shared mechanism, as real genetics does.

	paired contiguity gap (GA−random)			test accuracy	
N	fixed	scaled	denoise (r=8)	GA	random
12	+0.02	+0.02	+0.13	0.885	0.880
16	+0.06	+0.08	+0.19	0.882	0.877
20	+0.20	+0.20	+0.32	0.874	0.866
24	+0.14	+0.16	+0.19	0.866	0.862
28	+0.12	+0.17	+0.16	0.862	0.853

Table 1: Directed GA vs random at matched evaluations, 50 paired seeds (SEM ≈0.04 on each gap). The contiguity advantage is significant from N=20 and survives both a budget scaled with N and a denoising control (random given r=8 evaluations per mask, which if anything *widens* the gap). Test accuracy is tied throughout: the GA is marginally higher but always within the per-seed SD (≈0.04).

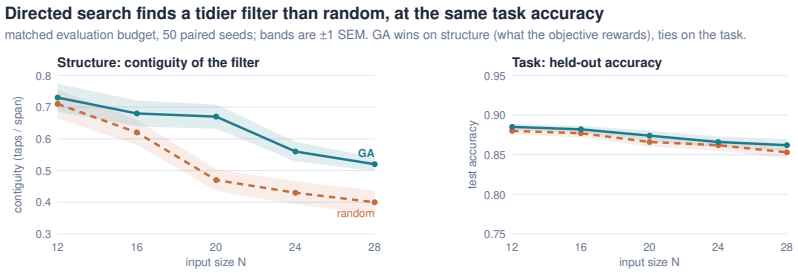

Figure 3: Directed search is *tidier, not better*. *Left:* filter contiguity, GA vs random at matched budget, the GA is clearly more contiguous. *Right:* held-out accuracy on the same seeds and the same axis honesty, the arms are tied. The GA wins on the structural term the objective rewards, not on the task. 50 paired seeds, bands ±1 SEM.

And it emerges from either direction. Run the same energy GA under grouped mutation from a *minimal* seed and grow up (a NEAT-style direction), and it reaches an essentially identical kernel to the dense seed pruned down (24 seeds); the emergence does not depend on the starting point. Grow-and-prune sparse training (SET, RigL) moves both ways too, but toward accuracy and sparsity; this *path-independence* of the emergent *structure*, one objective reaching it from an exhaustive and a minimal seed alike, we treat as a robustness check, not a novelty claim.

2.4 Directed search vs. random: tidier, not better

Does the *directed* energy search do real work, or would random sampling find as good a filter? The question is sharp because on a real benchmark our companion crossover study found evolution barely edges random, inside the noise. **Method.** At a *matched evaluation budget* we run the grouped-mutation GA against random offset masks (drawn across densities, kept by the same objective) over 50 *paired* seeds (same task per seed) at five input sizes N, and report both structure (contiguity) and task accuracy (Figure 3, Table 1).

Result: tidier, not better. The GA produces a markedly more contiguous filter than random from N=20 on (Table 1, each gap ≥3 SEM), *but the two arms are tied on held-out accuracy at every N*. So the GA is not a better network, it is a *tidier* one at equal performance, exactly what a directed optimizer should do on an objective it can climb: accuracy saturates without a tight kernel, so the only thing left to win is the energy term, which the GA descends by mutation where random can only sample. At scale it is *less diffuse than random*, not compact (on-relevance 0.21 at N=28); a compact filter truly emerges only at N=12.

The edge is direction, not denoising. The GA re-evaluates each survivor afresh every

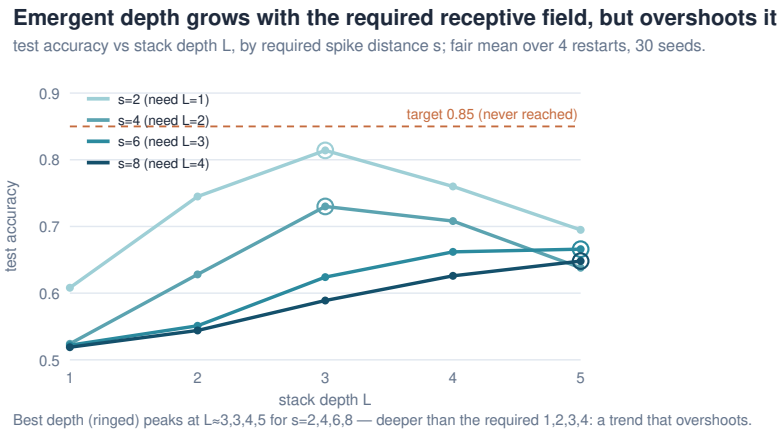

Figure 4: Emergent depth grows with the required receptive field but overshoots it. Under a fair mean over restarts, accuracy is near chance for stacks too shallow to span the spike pair and rises with depth, the optimum deepening with s; but it never reaches the 0.85 target and the best depth exceeds $s/2$ (the clean target-crossing staircase was a best-of-restarts artifact).

generation, so a marginal mask gets many attempts at the accuracy gate while random gets one. Give random r=8 evaluations per mask (best-of-r) at the same budget and the gap does not shrink, it *widens* (Table 1): best-of-r costs random distinct masks, and exploration matters more than noise-averaging. A budget scaled with N firms the large-N gap only slightly.

Why it matters. This *reconciles* the crossover null and confirms the project's founding conjecture, that a directed search beats random, under the condition it always assumed: the objective must have structure to climb. On NAS-Bench-101 it was flat near the top, so directed search tied random; here the energy objective is exploitable, so it climbs and wins. ("Tidier" is really *energy-efficient*, the more compressed structure at equal accuracy, so the GA is the better *search*.) The margin should widen with scale, where random covers a vanishing fraction of a space growing like 2^{2N} while directed search keeps climbing, which is why large-scale search uses evolution or gradients, not random; but at $N \leq 28$ that is a prediction, not a measurement.

2.5 Emergent depth grows with the receptive field, but overshoots it

Impose the feed-forward composition rule too, stack one reused block type, and the depth is the one free dimension. On a task that fires only when two spikes are exactly distance s apart (a unit must see both at once, receptive field $\geq s{+}1$, i.e. depth $\geq s/2$), accuracy sits near chance while the stack is too shallow to span the pair and rises with depth, and the depth of best accuracy grows with s (Figure 4). But, evaluated fairly (mean over restarts), the match is only *qualitative*: peak accuracy falls with s ($0.81, 0.73, 0.67, 0.65$ for s=$2, 4, 6, 8$) and never reaches the 0.85 target, so no depth is cleanly "selected," and the best depth *overshoots* the required $s/2$ ($L{\approx}3, 3, 4, 5$ versus $1, 2, 3, 4$). The clean target-crossing staircase we first reported was a best-of-restarts artifact; fairly measured, depth is a trend with the same overshoot the channel axis shows (Sec. 3.6).

3 The operators: reuse, recombination, translation

3.1 Reuse beats a free search at equal compute

Composing blocks explodes combinatorially, so we adopt the heuristic LeCun did: reuse one block type, stacked, and search only the depth. At *equal compute*, a free heterogeneous search (a GA over arbitrary block sequences, using a dilated-conv library) never solves more reliably

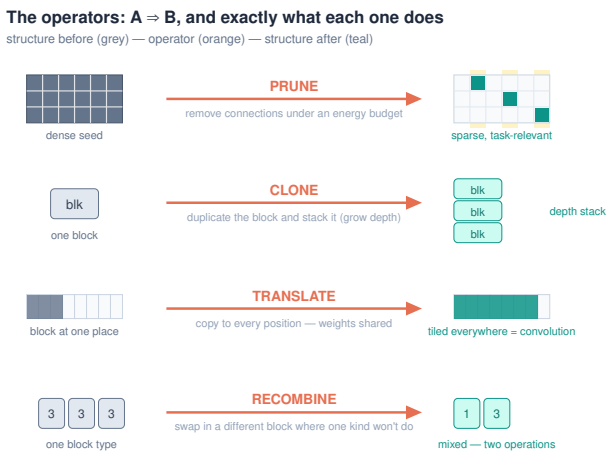

Figure 5: The four structural operators as $A \to B$, each with its exact action. Prune (remove connections under an energy budget), clone (duplicate and stack, for depth), translate (copy to every position with shared weights, a convolution), recombine (swap in a different block where one kind will not do). Each maps to a biological analog: pruning, gene/segment duplication, transposition/serial homology, sexual recombination. Of the four, only *prune* is applied inside the energy GA; *clone* and *translate* are studied as fixed shared-versus-unshared architectures (Sec. 3.1, 3.3), and *recombine* as a search over heterogeneous block sequences (mutation over sequences, not two-parent crossover).

than simply enumerating uniform reused blocks, and is cheaper only on the easiest task; on the harder tasks reuse is strictly better. Block diversity buys nothing but a larger search, the reuse heuristic is the tractable near-optimum, which is the user's intuition ("the combinations explode, so reuse the same block") made quantitative. This mirrors cell-based NAS (ENAS, DARTS), which search one cell and stack it.

3.2 Recombination is required when a task needs two different operations

Reuse breaks exactly where the biology predicts. On a task that needs a fine local motif $(+, -, +)$ at adjacent positions (visible only to a dilation-1 block) *and* a far spike at distance s (reached cheaply only by a dilation-3 block), no single reused block serves both roles: a uniform dilation-3 stack is blind to the fine motif (solving 0/8), a uniform dilation-1 stack cannot afford the reach. Recombination, a GA over mixed block sequences, solves 8/8 at about half the energy, and its solutions are literal two-operation mixes such as $[1, 3]$: one fine block, one coarse-reach block (Figure 6). Clone when scaling a working module; recombine only when you need a new kind of part, runners when the structure repeats, seeds when it must change.

3.3 Translation and scale: weight sharing is data-efficient (a reproduction)

The third operator, translation, replicating one working detector across positions with shared weights (transposition, serial homology, cortical-map tiling), is exactly convolution's tiling reached by replication. On a translation-invariant motif task a weight-shared filter (3 weights) beats a locally-connected baseline (one filter per position) at every training-set size, most when data is scarcest: the paired gap runs from +0.24 at 16 training examples to +0.14 at 128 (mean over restarts, ± 1 SD over 40 seeds). This is the textbook argument for weight sharing, one filter learns the motif from every position's examples while each independent filter starves, quantified with a fair estimator; a phenomenon of weight *sharing*, not architecture search (both arms are

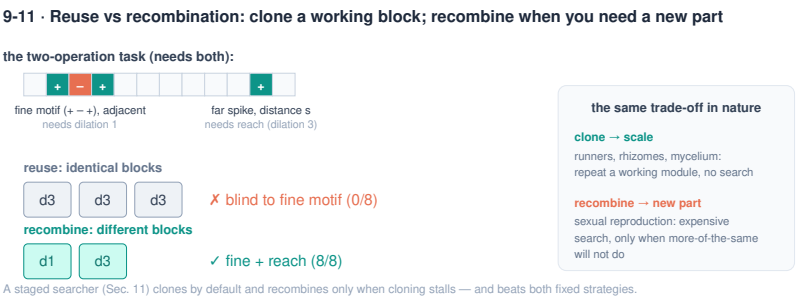

Figure 6: Reuse vs recombination, and the clone/recombine trade-off nature also makes. One reused block suffices for repetitive structure; a task needing two different operations forces recombination of different blocks.

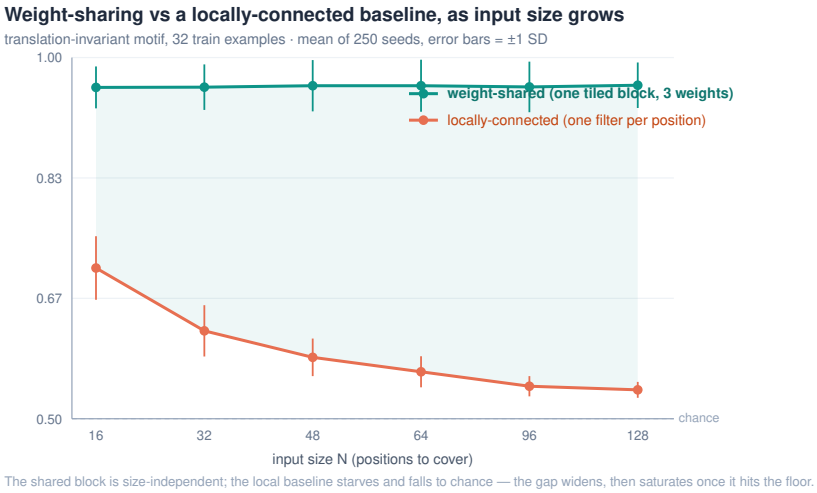

Figure 7: The weight-sharing advantage as input size grows (mean of 250 seeds, error bars ±1 SD). The shared block is size-independent; the locally-connected baseline starves and falls to chance. The gap widens, then saturates once the baseline hits the floor.

weight-tied vs untied, plain SGD). The advantage *grows with input size* (Figure 7, 250 seeds): the shared arm stays flat at ≈ 0.94 (size-independent) while the locally-connected baseline falls to chance and bloats to hundreds of weights; the gap widens then saturates (4–6× the SD, most growth at $N{=}16 \to 32$) and survives an oracle-supervised baseline ($+0.14$ at $N{=}16$ to $+0.38$ at $N{=}128$), so it is not a training-rule artifact.

3.4 The developmental chain: find a block, then clone and translate it

The operators above were each studied alone; here we *chain* the core of them in one run and compare a developmental trajectory against searching structure from scratch. On a translation-invariant motif task with scarce data (24 examples, 40 seeds), the trajectory is: (1) search from scratch, P independent per-position detectors trained from random; (2) find one stable shared block; (3) clone and translate it, the same filter tiled at every position (a convolution assembled by replication, not re-searched); (4) refine in place. Search-from-scratch *starves*, each position sees few examples, and reaches only 0.60; finding one block and tiling it reaches 0.86 ($+0.25$, $\sim$15 SEM of the paired mean). We label this honestly: the $+0.25$ is the *same* weight-sharing data-efficiency effect of Sec. 3.3 (one tiled block learns from every position while the P independent per-position detectors starve), now shown inside a chained pipeline, not a separate "reuse beats re-search" mechanism, and a fair *shared* baseline trained end-to-end on the same label captures

most of it. What the chain adds is the two side findings: refining the tiled copies in place *hurts* (0.84), because independent fine-tuning re-introduces per-position starvation and breaks the sharing that made reuse work; and annealing jitter hurts more (0.76), because reuse lands at the optimum, so perturbation only kicks it off, jitter is for escaping bad minima and reuse avoids them. This chains find, clone, and translate; whether the fourth operator, recombination of a *second* discovered block, *emerges* when the search must find the decomposition itself is the question of Sec. 3.5.

3.5 Does composition emerge when the search must find it? A boundary

The chain above, like the reuse and translation results, hands the search a decomposition and asks it to assemble the pieces. The sharper question is whether the *search itself*, given only the composite label and an energy budget, discovers the decomposition, both how many parts and which. We test it on a two-operation task (motif *A and B*, each at a random position; positive iff both present), where one weight-shared tiled channel caps at 0.75 by construction: sweep the channel count C under an energy budget (the energy-selection of Sec. 2.5, now on channels), train each candidate's shared filters and read-out jointly on the composite label *alone*, against an auxiliary-label *curriculum* (find A, find B, freeze, combine) as a supervised ceiling and an unshared per-position search as the floor (40 paired seeds).

Result: composition does not cleanly emerge without supervision. A one-operation task selects C^*=1; but on the two-operation task the two channels specialize to the two motifs in only 55% of runs (the rest collapse onto one), so C=2 plateaus at 0.81 (a fair shared baseline), the curriculum reaches 0.87, and the energy-selected count *overshoots* to C^*=3. Directed search under an energy budget does not, on its own, find the two-part decomposition, it underspecializes and pays for a redundant channel; the clean split appears only with per-operation supervision. This sharpens the contrast between the axes: emergent *depth* at least grows with the receptive field from the task label alone (Sec. 2.5, even if it overshoots), whereas emergent *composition* on the channel axis does not organize at all without per-operation supervision. It also explains a chain that only *appears* to recombine, hand it the decomposition and its gain is weight sharing plus that supervision, not emergent composition.

4 Related work, and how we differ

The closest neighbors, and the honest distinction: **DARTS** starts from an exhaustive supernet and prunes to a final architecture, closest in spirit, but via continuous gradient relaxation over predefined operations, not an energy GA, and it never asks whether convolution emerges. **Optimal Brain Damage** and the **Lottery Ticket Hypothesis** prune dense to sparse, but by gradient magnitude, and find sparse subnetworks, not convolutions. **NEAT**, **HyperNEAT**, and Cellular Encoding evolve topology but *grow from a minimal seed*, the opposite direction to pruning from exhaustive. **Cell-based NAS** (ENAS, regularized evolution) reuses one cell and stacks it, which our reuse observations echo. The essential difference is the *use* of the GA: evolutionary NAS runs it as an *optimizer* to find a performant architecture, whereas we run it as a *lens* to characterize which pieces of convolution emerge, and which refuse, under an energy budget from an exhaustive seed, a use of evolution we are not aware of in the literature. That characterization, and its negatives, is the claim, not the mechanisms, which are known.

Two framings place this experiment in a longer line. The view of network topology as problem-specific inductive bias that must be engineered by hand goes back decades [11]; here we ask the complementary question, whether that structure can instead *emerge* from the energy budget rather than be built in.[2] Viewed more broadly, the energy budget is a description-length

[2]The broader claim that such emergence is substrate-independent is argued informally in a companion essay [12].

pressure: the emergent weight-shared filter is a compressor whose inductive bias, locality and sharing, matches the task's translation symmetry, an instance of the evolutionary-search route to compressor discovery set out in [10].

5 Limitations

The tasks are small synthetic constructions built so the intended structure is optimal; they demonstrate what emerges under an energy budget on a task shaped like convolution, and do not by themselves generalize to real data. The reuse and weight-sharing results are a property of *fixed* shared-versus-unshared architectures (data efficiency under SGD), not evidence about what the evolutionary search discovers; we label them accordingly. The developmental chain (Section 3.4) covers find, clone, and translate; when the search must *discover* the recombination of a second block it does not cleanly do so (Sec. 3.5), so a real dataset and scale, not another operator, are the natural next steps. What limits this work is scale, not the absence of an idea: the new parts, the sharing/energy decoupling and the directed-vs-random reconciliation, are real but small, and the tasks are toy, so its natural home is a workshop, reproducibility, or explainer track rather than a top venue, where the bar is significance at scale.

6 Conclusion

Impose the domain's real symmetries; a few biological operators, studied here one at a time, could then assemble the rest. Under an energy budget on an exhaustive seed, sparsity and feature selection emerge and weight sharing is adopted, and a compact convolution emerges too, but only once mutation acts on the shared feature rather than the single edge, for a decoupling reason we can state in a line. On top of imposed priors, depth grows with the receptive field (overshooting it), reuse beats a free search at equal compute, recombination is required only when a task needs a genuinely different operation, and weight-shared tiling is data-efficient at a degree that grows with scale then saturates. Where the search must *discover* a compositional decomposition itself it does not, the parts failing to specialize without supervision (Sec. 3.5), a boundary we mark rather than paper over. The contribution is the honest map, the negatives, and the reproducibility; every number regenerates from one command.

References

[1] Y. LeCun, J. S. Denker, S. A. Solla. Optimal Brain Damage. *NeurIPS* 1990.
[2] Y. LeCun et al. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation* 1(4), 1989.
[3] J. Frankle, M. Carbin. The Lottery Ticket Hypothesis. *ICLR* 2019.
[4] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.
[5] H. Pham, M. Guan, B. Zoph, Q. Le, J. Dean. Efficient Neural Architecture Search via Parameter Sharing. *ICML* 2018.
[6] K. O. Stanley, R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2), 2002.
[7] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.
[8] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi. Optimization by Simulated Annealing. *Science* 220, 1983.
[9] V. Gavrilov. SMBPANN reference implementation, 2001–2015–2026. `https://github.com/Anode1/SMBPANN`.
[10] V. Gavrilov. Intelligence Is the Discovery of Compressors. 2026. `https://doi.org/10.5281/zenodo.20440111`.
[11] V. Gavrilov. Backpropagation Feed-Forward Neural Networks. Seminar thesis, Open University of Israel, 1997 (digitized 2026). `https://doi.org/10.5281/zenodo.20450526`.
[12] V. Gavrilov. Emergence Does Not Care About Substrate. Companion essay, 2026. `https://gavr144.substack.com/p/emergence-does-not-care-about-substrate`.